

计算机科学与技术学院

系统开发工具基础

第四周实验报告

学生姓名： 杜铭昊

学号： 25020007021

授课教师： 周小伟

实验环境： Ubuntu Linux 虚拟机

报告内容： 课堂第 13-16 题与 10 个扩展实例

实验日期： 2026 年 9 月 14 日

代码质量、Make 构建、API 处理与综合交付

目录

1 实验环境与证据规范	3
2 第 13 题：建立可执行的本地质量门禁	3
2.1 实验原理	3
2.2 步骤 1：复制并配置项目	3
2.3 步骤 2：补充行为测试	3
2.4 步骤 3：格式化、检查与测试	4
2.5 步骤 4：编写统一检查入口	4
2.6 验收结果	4
2.7 结果与反思	4
3 第 14 题：让 Make 只重建真正受影响的产物	5
3.1 实验原理	5
3.2 步骤 1：建立源文件	5
3.3 步骤 2：编写准确 Makefile	5
3.4 步骤 3：验证首次构建和无操作构建	5
3.5 步骤 4：验证依赖传播与清理范围	6
3.6 验收结果	6
3.7 结果与错误反思	6
4 第 15 题：把本地 API 数据转换为可读报告	7
4.1 实验原理	7
4.2 步骤 1：启动本地服务并获取数据	7
4.3 步骤 2：筛选与排序	7
4.4 步骤 3：生成 summary.md	7
4.5 验收结果	8
4.6 结果与错误反思	8
5 第 16 题：修复并交付陌生的小型工具仓库	9
5.1 实验原理	9
5.2 步骤 1：制造并确认真实功能失败	9
5.3 步骤 2：定位并实施最小修复	9
5.4 步骤 3：建立 Make 交付入口	9
5.5 步骤 4：检查、构建与摘要	10
5.6 验收结果	10
5.7 结果与错误反思	10
6 第四周扩展实例 1–10	11
6.1 实例 1：ruff 格式门禁	11
6.2 实例 2：静态检查自动修复	11
6.3 实例 3：参数化测试	12
6.4 实例 4：最小 Make 增量构建	12
6.5 实例 5：Make 干运行与依赖链	13
6.6 实例 6：并行 Make	13
6.7 实例 7：curl 错误退出契约	14
6.8 实例 8：jq 聚合统计	14

6.9 实例 9: CLI JSON 输出契约	15
6.10 实例 10: 完整发布流水线	15
7 GitHub 分阶段提交记录	16
8 综合结论	17

1 实验环境与证据规范

实验在 Ubuntu 虚拟机中实测，Windows 用于文件同步和 GitHub 推送。主要工具包括 Python 3.12、ruff 0.16.5、pytest、GNU Make、curl、jq、build、Git 和 SHA-256 工具。虚拟机系统默认 Python 版本较旧，因此涉及 f-string 的步骤显式使用课程 Python 3.12。所有题目均保存源码、运行脚本、结果日志和 Linux 桌面截图；扩展实例每两个形成一次独立提交。

2 第 13 题：建立可执行的本地质量门禁

2.1 实验原理

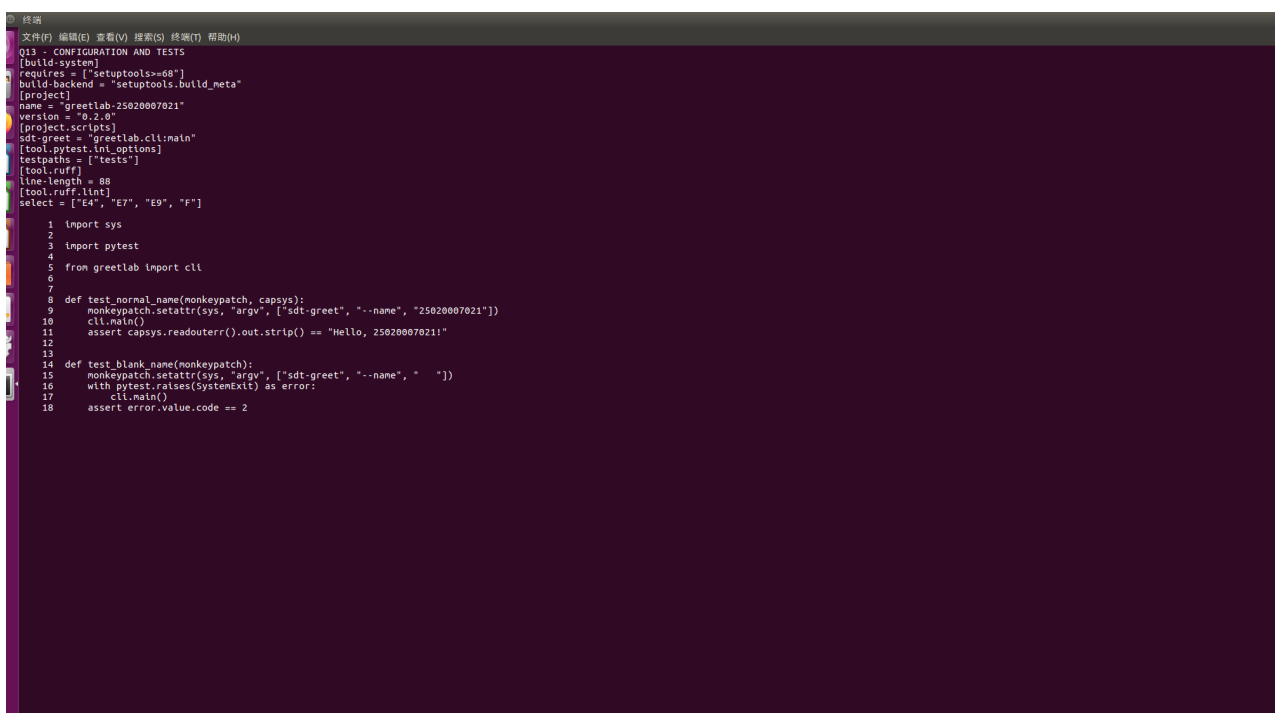
质量门禁把格式、静态检查和自动化测试组合为单一、可重复执行的入口。这三个层次分别回答“代码排版是否统一”“源码是否存在可静态识别的问题”“程序行为是否符合需求”。`ruff format --check` 只检查格式而不修改文件；`ruff check` 检测未使用导入、语法错误等问题；`pytest` 通过断言验证正常输入和异常输入。检查脚本采用 `set -euo pipefail`：任一命令返回非零状态，脚本立即停止并把失败传递给调用者，因此本地执行、Make 和后续持续集成都能使用同一套通过标准。

2.2 步骤 1：复制并配置项目

以第 10 题 `greetlab` 为基础建立 `q13`，保留 `src/greetlab` 源码布局 and `tests` 测试目录。在 `pyproject.toml` 中配置 `pytest` 测试目录、`ruff` 行宽以及 E4、E7、E9、F 规则，不使用全局忽略。这样工具从项目配置中读取规则，换一台机器进入项目后仍可按相同标准检查。

2.3 步骤 2：补充行为测试

正常姓名测试把命令行参数设为学号，调用入口函数后捕获标准输出，并严格断言为 `Hello, 25020007021!`。异常测试传入只含空格的姓名，断言程序抛出 `SystemExit` 且退出码为 2。前者验证主要功能，后者验证输入边界和错误契约，两个测试均包含可判定结果的断言。



```
q13 - CONFIGURATION AND TESTS
[build-system]
requires = ["setuptools>=68"]
build-backend = "setuptools.build_meta"
[project]
name = "greetlab-25020007021"
version = "0.2.0"
[project.scripts]
sdt-greet = "greetlab.cli:main"
[tool.pytest.ini_options]
testpaths = ["tests"]
[tool.ruff]
line-length = 88
[tool.ruff.lint]
select = ["E4", "E7", "E9", "F"]

1 import sys
2
3 import pytest
4
5 from greetlab import cli
6
7 def test_normal_name(monkeypatch, capsys):
8     monkeypatch.setattr(sys, "argv", ["sdt-greet", "--name", "25020007021"])
9     cli.main()
10     assert capsys.readouterr().out.strip() == "Hello, 25020007021!"
11
12
13 def test_blank_name(monkeypatch):
14     monkeypatch.setattr(sys, "argv", ["sdt-greet", "--name", ""])
15     with pytest.raises(SystemExit) as error:
16         cli.main()
17     assert error.value.code == 2
```

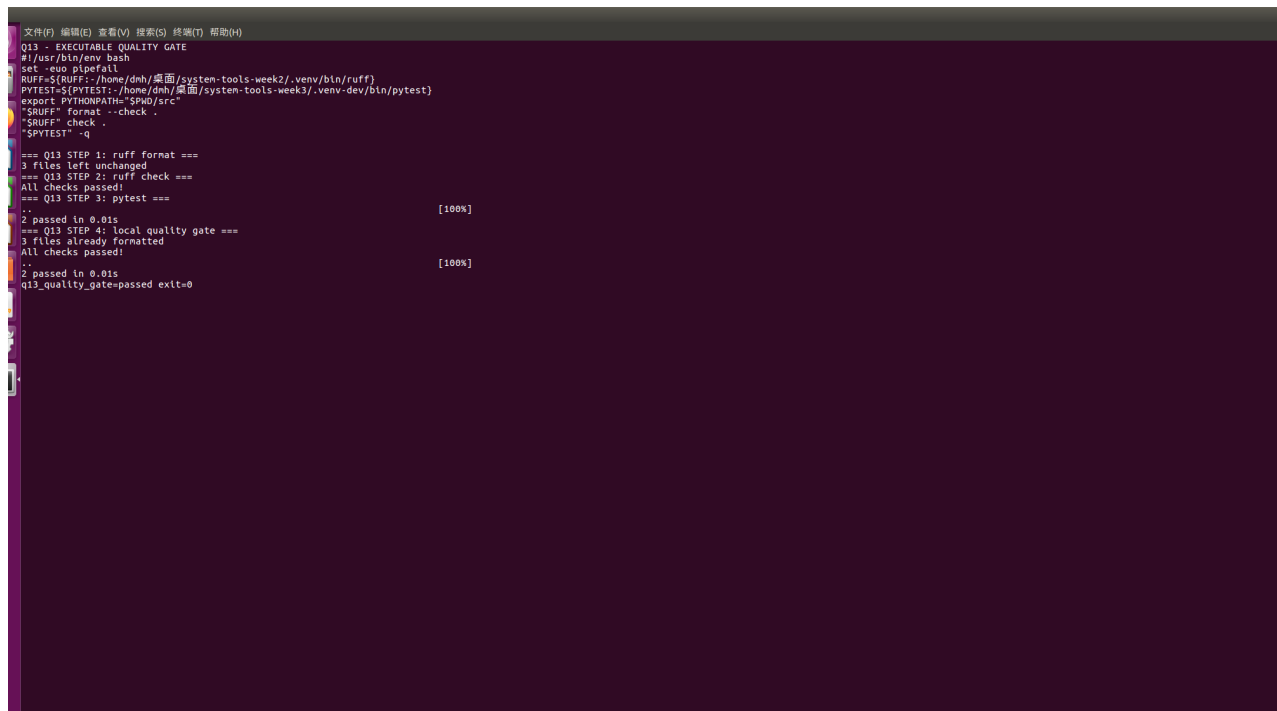
图 1：第 13 题：项目配置与正常/空白姓名测试

2.4 步骤 3：格式化、检查与测试

先执行 `ruff format .` 统一现有代码格式, 再依次运行 `ruff format --check .`、`ruff check .` 和 `pytest -q`。实测结果为 3 个文件格式正确、静态检查全部通过、2 个测试通过。把“自动修改格式”和“只检查格式”分开, 可避免门禁在无人确认的情况下改写源码。

2.5 步骤 4：编写统一检查入口

创建可执行脚本 `check.sh`, 按“格式检查 → 静态检查 → 自动测试”的顺序调用三个工具, 并只在全部成功后输出 `q13_quality_gate=passed`。实际运行脚本返回码为 0, 说明整个质量门禁通过, 而不是某一个工具单独通过。



```
Q13 - EXECUTABLE QUALITY GATE
#!/usr/bin/env bash
set -euo pipefail
RUFF=$(RUFF:-/home/dnh/桌面/system-tools-week2/.venv/bin/ruff)
PYTEST=$(PYTEST:-/home/dnh/桌面/system-tools-week3/.venv-dev/bin/pytest)
export PYTHONPATH="$PWD/src"
"SRUFF" format --check .
"SRUFF" check .
"SPYTEST" -q

=== Q13 STEP 1: ruff format ===
3 files left unchanged
=== Q13 STEP 2: ruff check ===
All checks passed!
=== Q13 STEP 3: pytest ===
2 passed in 0.01s [100%]
=== Q13 STEP 4: local quality gate ===
3 files already formatted
All checks passed!
2 passed in 0.01s [100%]
q13_quality_gate=passed exit=0
```

图 2: 第 13 题: `check.sh` 与质量门禁通过结果

2.6 验收结果

验收项	实测结果	判定
格式与静态检查	3 个文件格式正确, ruff 通过	通过
行为测试	2 passed	通过
统一入口	退出码 0 并输出 passed	通过

2.7 结果与反思

本题所有门禁通过。单独执行工具容易遗漏步骤, 统一脚本使开发者和持续集成环境执行同一份质量契约; 不应通过全局忽略规则来“消除”错误, 而应修复根因。

3 第 14 题：让 Make 只重建真正受影响的产物

3.1 实验原理

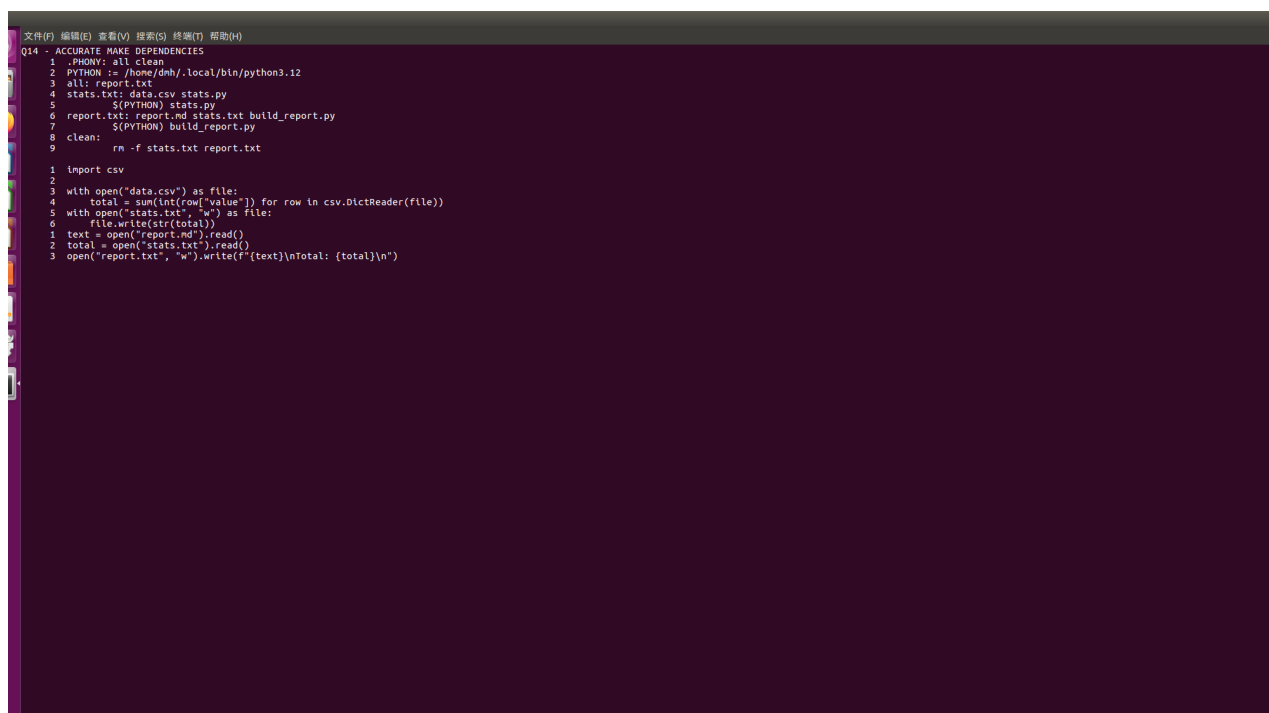
Make 把构建过程表示为有向依赖图，并比较目标与依赖文件的修改时间。若目标不存在，或任一依赖比目标新，就执行对应配方。本题的依赖链为“data.csv + stats.py -> stats.txt”以及“report.md + stats.txt + build_report.py -> report.txt”。因此数据改变时先重算统计，再重新生成报告；所有输入不变时不应执行任何配方。all 和 clean 表示动作而非真实文件，必须声明为 .PHONY，避免同名文件导致命令被错误跳过。

3.2 步骤 1：建立源文件

创建题目给定的 data.csv、stats.py、build_report.py 和 report.md。数据值 2、3、5 的合计应为 10。

3.3 步骤 2：编写准确 Makefile

默认目标 all 依赖 report.txt；统计目标依赖数据和统计脚本；报告目标依赖 Markdown、统计结果和报告脚本；clean 只删除两个生成物。由于系统默认 Python 无法解析题目中的 f-string，Makefile 明确指定/home/dmh/.local/bin/python3.12。



```
Q14 - ACCURATE MAKE DEPENDENCIES
1 .PHONY: all clean
2 PYTHON := /home/dmh/.local/bin/python3.12
3 all: report.txt
4 stats.txt: data.csv stats.py
5     $(PYTHON) stats.py
6 report.txt: report.md stats.txt build_report.py
7     $(PYTHON) build_report.py
8 clean:
9     rm -f stats.txt report.txt

1 import csv
2
3 with open("data.csv") as file:
4     total = sum(int(row["value"]) for row in csv.DictReader(file))
5 with open("stats.txt", "w") as file:
6     file.write(str(total))
7 text = open("report.md").read()
8 total = open("stats.txt").read()
9 open("report.txt", "w").write(f"{text}\nTotal: {total}\n")
```

图 3：第 14 题：Makefile 依赖关系与处理脚本

3.4 步骤 3：验证首次构建和无操作构建

第一次执行 make 时目标文件均不存在，Make 依次运行统计和报告配方，得到总计 10 和最终报告。紧接着再次执行 make，终端显示没有需要执行的任务，两个产物时间戳也没有变化，说明 Make 确实复用了已有结果。

3.5 步骤 4：验证依赖传播与清理范围

执行 `touch data.csv` 只更新上游数据时间戳，再次 `make` 时 `stats.txt` 先重建，随后依赖它的 `report.txt` 重建，两个时间戳严格递增。最后执行 `make clean`，确认 `stats.txt` 和 `report.txt` 被删除，而 CSV、Python 脚本、Markdown 源文件与 Makefile 仍保留。

```

Q14 - FIRST, NO-OP, TOUCH AND CLEAN RESULTS
=== Q14 STEP 1: clean and first make ===
rm -f stats.txt report.txt
/home/dmh/.local/bin/python3.12 stats.py
/home/dmh/.local/bin/python3.12 build_report.py
products:
108 Data Report

Total: 18
=== Q14 STEP 2: second make ===
make: Nothing to be done for 'all'.
=== Q14 STEP 3: touch data.csv and make ===
/home/dmh/.local/bin/python3.12 stats.py
/home/dmh/.local/bin/python3.12 build_report.py
=== Q14 STEP 4: dependency timestamps ===
2026-09-04 00:00:39.594560925 -0700 data.csv
2026-09-04 00:00:39.516566874 -0700 stats.txt
2026-09-04 00:00:39.524566855 -0700 report.txt
=== Q14 STEP 5: clean only generated files ===
rm -f stats.txt report.txt
q14_incremental_build=passed
  
```

图 4：第 14 题：首次构建、无操作、touch 重建和 clean 结果

3.6 验收结果

场景	预期行为	实测
首次 make	两级目标均生成	通过
输入不变	不重复构建	通过
touch 数据	两级依赖依次重建	通过
make clean	只删除生成物	通过

3.7 结果与错误反思

增量构建验证通过。首次运行出现 `SyntaxError`，根因是系统默认 Python 过旧而非 Make 依赖错误；修复方法是将解释器作为显式构建变量，避免“同名命令、不同运行时”导致不可复现。

4 第 15 题：把本地 API 数据转换为可读报告

4.1 实验原理

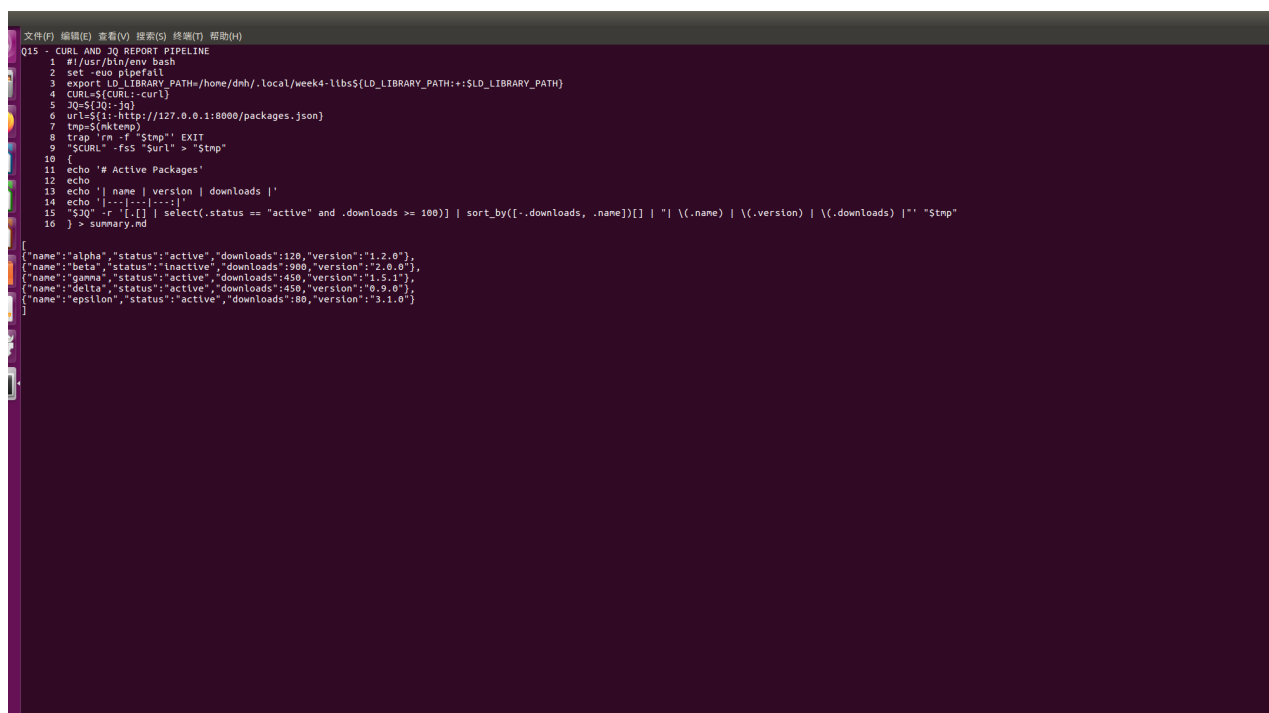
本题形成“HTTP 获取 → JSON 校验 → 条件筛选 → 稳定排序 → Markdown 呈现”的数据流水线。curl 的 -f 使 HTTP 4xx/5xx 转化为非零退出码，-sS 隐藏进度条但保留错误信息；jq 按 JSON 对象和字段操作，避免 grep、awk 等文本切割破坏结构。筛选条件是 status == active 且下载量不少于 100；排序时先按 downloads 降序，再按 name 升序处理并列，从而保证结果稳定、可复现。

4.2 步骤 1：启动本地服务并获取数据

保存包含 5 条软件包记录的 JSON，在 q15 目录启动仅供本机实验使用的 HTTP 服务。脚本记录服务进程号，并通过 trap 保证退出时终止服务、删除临时响应文件。随后 curl 访问 127.0.0.1:8000，jq 读取数组长度为 5；这一步同时验证了端口、URL、HTTP 响应和 JSON 语法。

4.3 步骤 2：筛选与排序

将 HTTP 响应交给 jq，先选择 status 为 active 且 downloads 不少于 100 的对象，再构造仅含 name、version、downloads 的输出。排序结果为 delta、gamma、alpha；其中 delta 与 gamma 下载量均为 450，二级键 name 升序使 delta 排在 gamma 之前，alpha 以 120 排在最后。



```
Q15 - CURL AND JQ REPORT PIPELINE
1 #!/usr/bin/env bash
2 set -euo pipefail
3 export LD_LIBRARY_PATH=/home/dnh/.local/week4-libs:${LD_LIBRARY_PATH}
4 CURL=$(curl -sS)
5 JQ=$(jq -c)
6 url=$(http://127.0.0.1:8000/packages.json)
7 tmp=$(mktemp)
8 trap 'rm -f "$tmp" EXIT'
9 "$CURL" -fs "$url" > "$tmp"
10 {
11 echo '# Active Packages'
12 echo
13 echo 'name | version | downloads |'
14 echo '---|---|---|'
15 "$JQ" -r '[.[] | select(.status == "active" and .downloads >= 100)] | sort_by([-.downloads, .name])[] | "\(.name) | \(.version) | \(.downloads)"' "$tmp"
16 } > summary.md
```

```
{
  "name": "alpha", "status": "active", "downloads": 120, "version": "1.2.0",
  "name": "beta", "status": "inactive", "downloads": 900, "version": "2.0.0",
  "name": "gamma", "status": "active", "downloads": 450, "version": "1.5.1",
  "name": "delta", "status": "active", "downloads": 450, "version": "0.9.0",
  "name": "epsilon", "status": "active", "downloads": 180, "version": "3.1.0"
}
```

图 5: 第 15 题：curl、jq 与 Markdown 生成脚本

4.4 步骤 3：生成 summary.md

api_report.sh 先写入 Markdown 标题和表头，再把 jq 生成的三列数据逐行转换为表格。最终 summary.md 依次包含 delta 0.9.0 450、gamma 1.5.1 450、alpha 1.2.0 120。最后检查文件内容和脚本退出码，确认生成物完整且流水线成功结束。

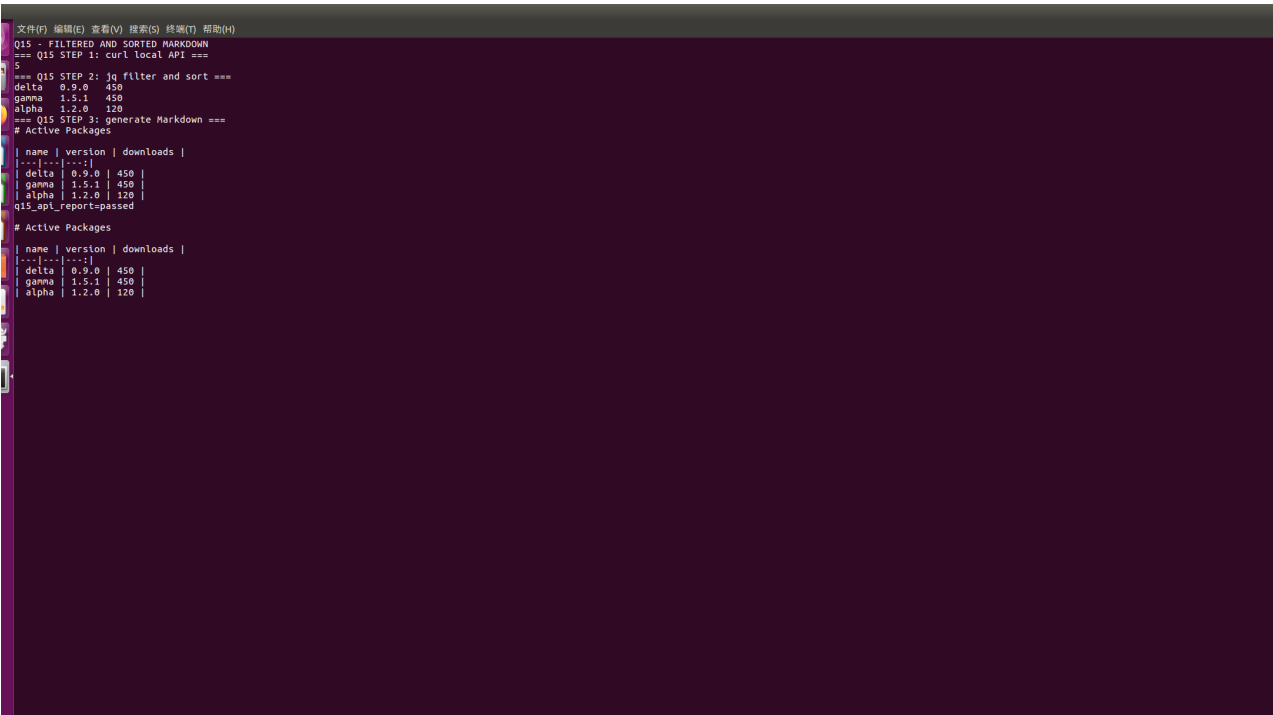


图 6: 第 15 题：过滤排序结果与 summary.md

4.5 验收结果

验收项	实测结果	判定
HTTP 与 JSON	数组长度 5	通过
过滤	保留 3 条合格记录	通过
排序	delta、gamma、alpha	通过
报告	Markdown 三列表格	通过

4.6 结果与错误反思

API 报告生成通过。虚拟机最初缺少 curl 和 jq，且无无密码 sudo 权限；因此从 Ubuntu 官方软件源下载相应软件包并仅解压到用户目录，未改变系统级配置。脚本显式设置工具路径和 jq 动态库路径，保证他人可以理解运行环境。

5 第 16 题：修复并交付陌生的小型工具仓库

5.1 实验原理

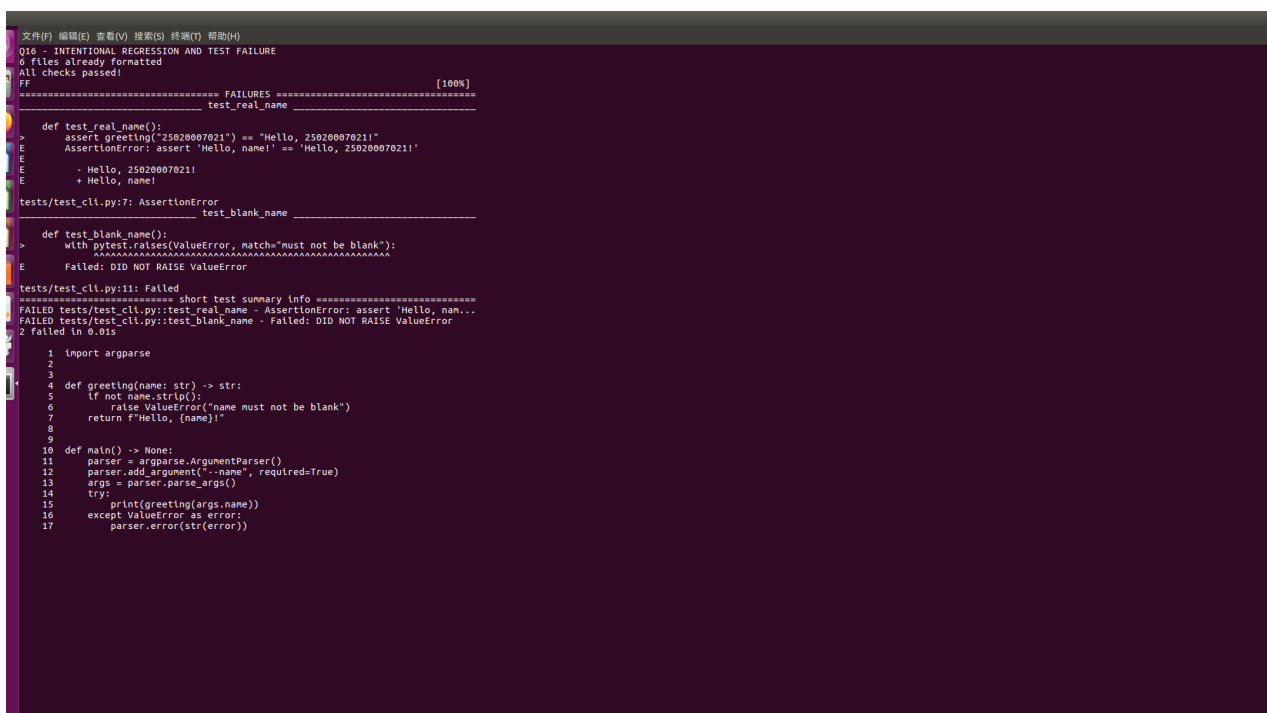
综合交付遵循“复现缺陷 → 用测试固定现象 → 最小修复 → 全量回归 → 构建制品 → 计算摘要”的闭环。测试失败是修复前的重要基线，只有同一测试在修复后转为通过，才能证明修改针对了真实问题。Makefile 把 check 设为 build 的前置条件，从依赖结构上保证“不通过检查就不构建”。wheel 是可安装的 Python 分发制品，SHA-256 摘要把文件内容映射为固定指纹，可用于传输后的完整性校验。

5.2 步骤 1：制造并确认真实功能失败

复制 q13 为 q16 并先阅读项目结构、入口函数、测试和配置。为验证测试确实能够发现功能回归，把问候函数临时改为始终返回字面量 `Hello, name!`。先格式化这份故障代码，使格式门禁通过，再执行统一检查。结果 ruff 通过而 pytest 明确报告 2 项失败：真实姓名未被代入，空白姓名也未抛出预期异常；总退出码为 1。由此确认测试覆盖了题目关心的两类行为。

5.3 步骤 2：定位并实施最小修复

根据失败断言定位到问候函数，而不是修改测试来迎合错误实现。修复时先去除输入两端空白；若结果为空则抛出 `ValueError`，CLI 捕获后转换为参数错误；否则用清洗后的真实参数构造问候语。修改范围只涉及缺陷路径，没有更名、重排目录或进行无关重构，降低了引入新回归的风险。



```
Q16 - INTENTIONAL REGRESSION AND TEST FAILURE
0 files already formatted
All checks passed!
FF
===== FAILURES =====
test_real_name
def test_real_name():
    assert greeting("25020007021") == "Hello, 25020007021!"
AssertionError: assert 'Hello, name!' == 'Hello, 25020007021!'
- Hello, 25020007021!
+ Hello, name!
tests/test_cli.py:7: AssertionError
test_blank_name
def test_blank_name():
    with pytest.raises(ValueError, match="must not be blank"):
        greeting("")
Failed: DID NOT RAISE ValueError
tests/test_cli.py:11: Failed
===== short test summary info =====
FAILED tests/test_cli.py::test_real_name - AssertionError: assert 'Hello, nam...
FAILED tests/test_cli.py::test_blank_name - Failed: DID NOT RAISE ValueError
2 failed in 0.01s

1 import argparse
2
3
4 def greeting(name: str) -> str:
5     if not name.strip():
6         raise ValueError("name must not be blank")
7     return f"Hello, {name}!"
8
9
10 def main() -> None:
11     parser = argparse.ArgumentParser()
12     parser.add_argument("--name", required=True)
13     args = parser.parse_args()
14     try:
15         print(greeting(args.name))
16     except ValueError as error:
17         parser.error(str(error))
```

图 7：第 16 题：故意回归、测试失败与最终实现

5.4 步骤 3：建立 Make 交付入口

Makefile 包含 check、build、clean 三个伪目标。check 调用统一质量脚本；build 显式依赖 check，只有格式、静态检查和测试全部通过后，才清理旧制品并用 Python 3.12 调用 build 模块；clean 只移除 build、dist 和元数据目录。这样质量验证、构建与清理都具有明确入口和退出状态。

5.5 步骤 4：检查、构建与摘要

先执行 `make check`，得到格式检查通过、`ruff` 通过和 2 个测试通过；再执行 `make build`，它会再次经过 `check` 前置条件并成功生成 0.3.0 版 wheel。随后对唯一的 wheel 计算 SHA-256 并保存摘要，最后核对文件名、文件存在性和摘要输出，形成可追溯的交付证据。

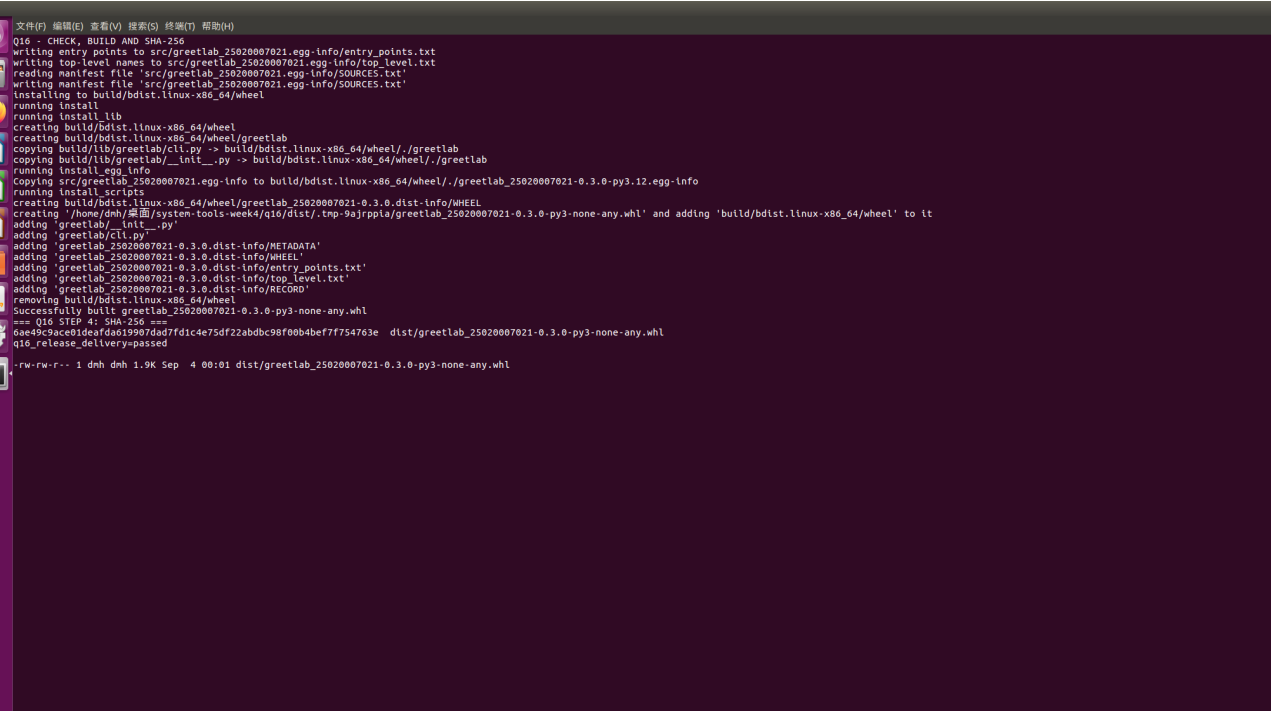


图 8: 第 16 题：质量检查、wheel 构建与 SHA-256 结果

5.6 验收结果

阶段	关键证据	判定
修复前	2 项功能测试失败，退出码 1	缺陷已复现
修复后	ruff 通过，2 passed	回归通过
构建	wheel 生成成功	通过
完整性	SHA-256 摘要已保存	通过

5.7 结果与错误反思

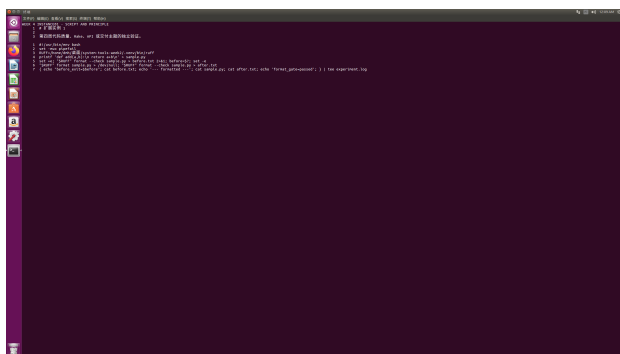
综合交付链通过。最初的故障演示被格式检查提前拦截，无法证明测试确实捕获功能缺陷；调整为先格式化故障代码后再执行 `check`，最终观察到两个真实功能测试失败。说明验证实验时必须确保失败来自目标缺陷，而不是更早的无关门禁。

6 第四周扩展实例 1-10

扩展实例围绕代码质量、自动测试、Make 依赖、并行构建、HTTP 错误契约、jq 统计和发布流水线设计。每个实例均在 Ubuntu 执行，并保存脚本与结果截图。

6.1 实例 1: ruff 格式门禁

故意创建缩进和逗号格式不规范的函数，`ruff format --check` 先返回 1 并显示差异；执行格式化后再次检查通过。原理是把代码风格转换为可执行约束。



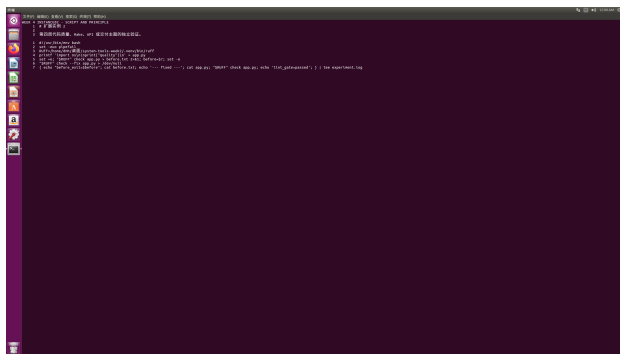
实例 1: 脚本与步骤



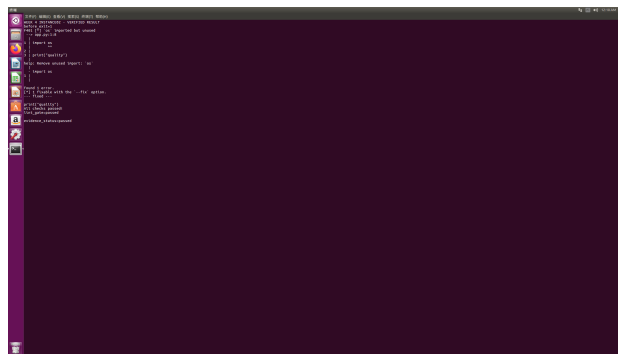
实例 1: 失败、修复和通过

6.2 实例 2: 静态检查自动修复

在程序中加入未使用的 `import os`, ruff 报告 F401 并返回 1；使用安全修复删除导入，最终 All checks passed。



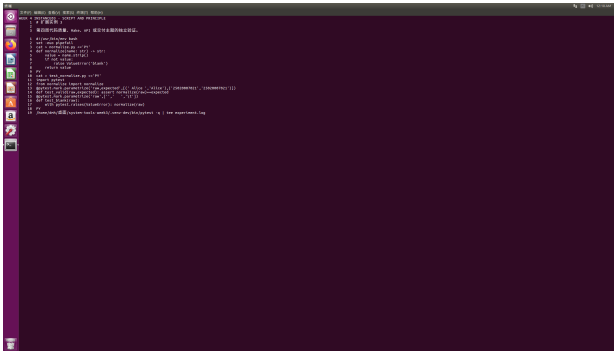
实例 2: 静态检查脚本



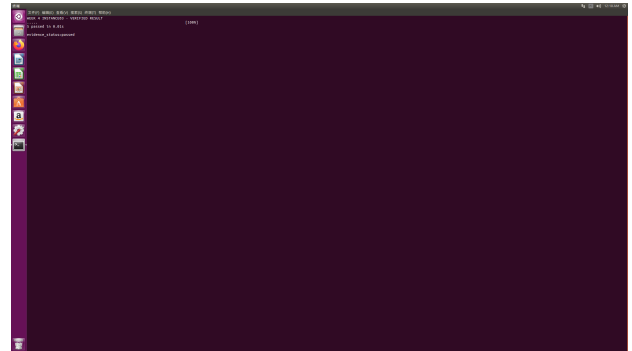
实例 2: F401 与修复结果

6.3 实例 3：参数化测试

实现姓名去空格与空值拒绝，通过 `pytest` 参数化覆盖 2 个正常输入和 3 个异常输入，最终 5 个测试全部通过。参数化可在不复制测试结构的情况下扩展边界覆盖。



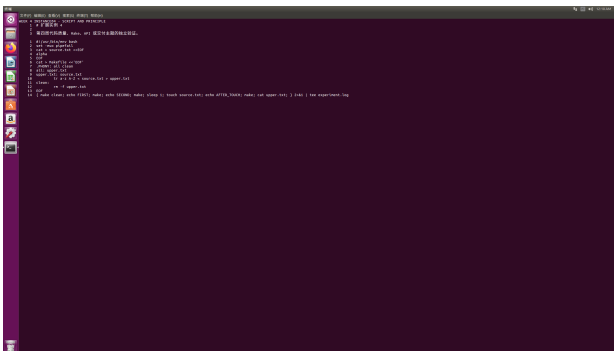
实例 3：参数化测试过程



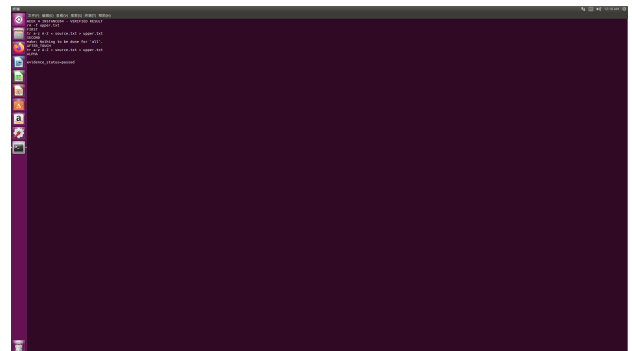
实例 3：5 项测试通过

6.4 实例 4：最小 Make 增量构建

首次生成大写文件，第二次 `make` 不执行配方；`touch` 源文件后目标重新生成并得到 ALPHA，证明时间戳依赖生效。



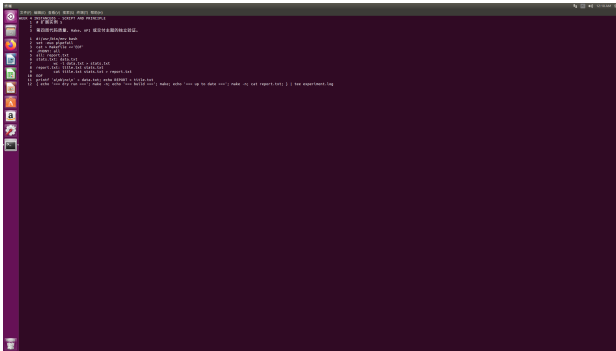
实例 4：Makefile 与步骤



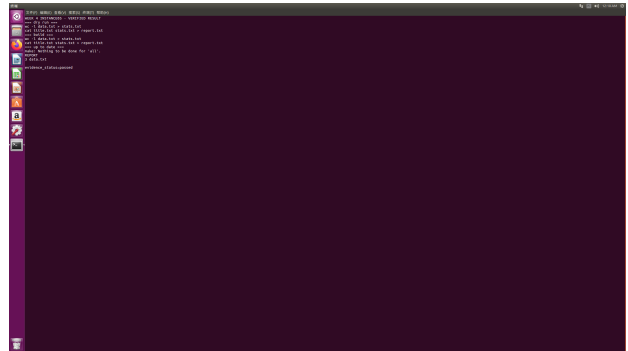
实例 4：增量构建结果

6.5 实例 5: Make 干运行与依赖链

使用 `make -n` 预览两条配方，正式构建后再次干运行显示无任务；报告正确包含标题和 3 行统计结果。干运行适合在不改动文件时审计构建计划。



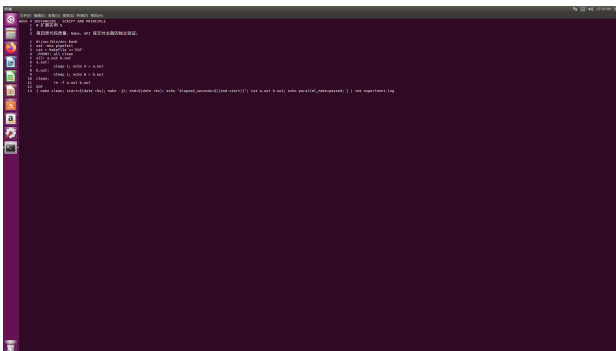
实例 5: 依赖链脚本



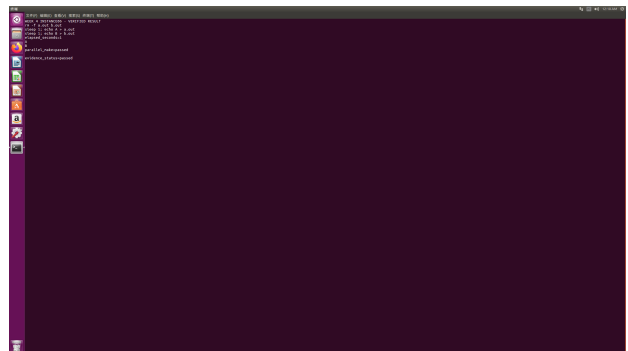
实例 5: 干运行和产物

6.6 实例 6: 并行 Make

两个互不依赖目标各等待 1 秒，`make -j2` 并行执行，总耗时约 1 秒且生成 A、B 两个文件，说明依赖图也决定可并行程度。



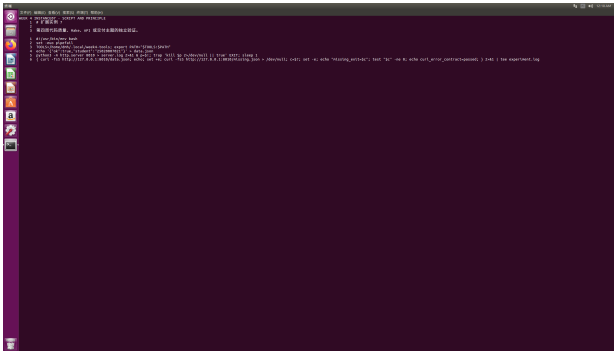
实例 6: 并行目标



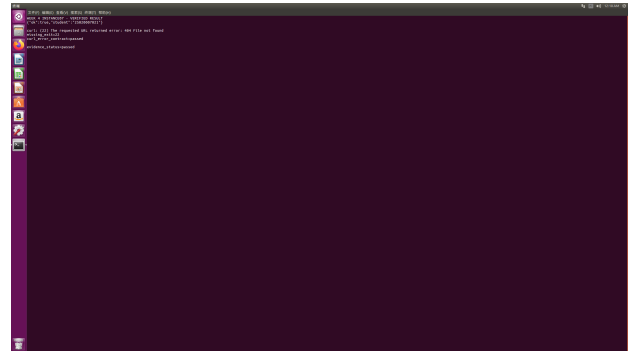
实例 6: 并行结果

6.7 实例 7: curl 错误退出契约

本地服务正常资源返回 JSON；不存在资源因 `-f` 得到 HTTP 404 和退出码 22，脚本断言其非零。网络脚本不仅要验证成功路径，也要让错误可被上层自动识别。



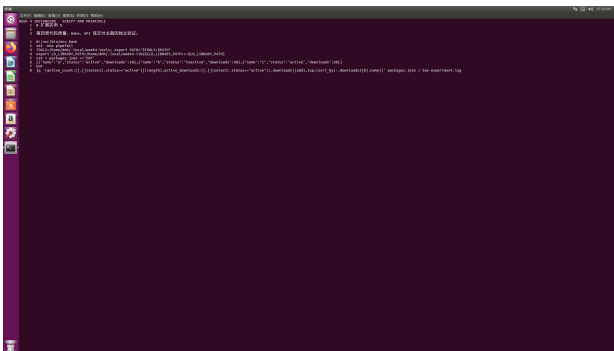
实例 7: HTTP 测试脚本



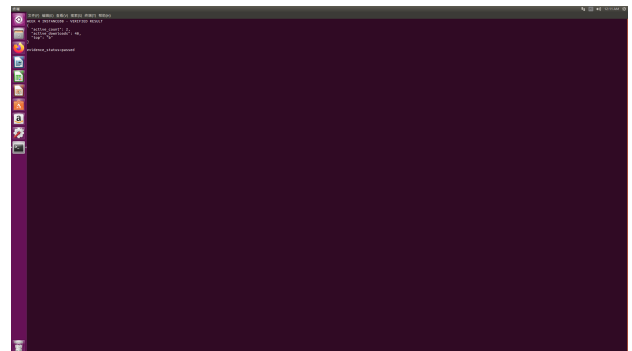
实例 7: 正常响应与 404 退出码

6.8 实例 8: jq 聚合统计

对 JSON 一次性计算活动记录数 2、活动下载总数 40 和总体下载最高项 b。结构化聚合避免多次解析或手工统计。



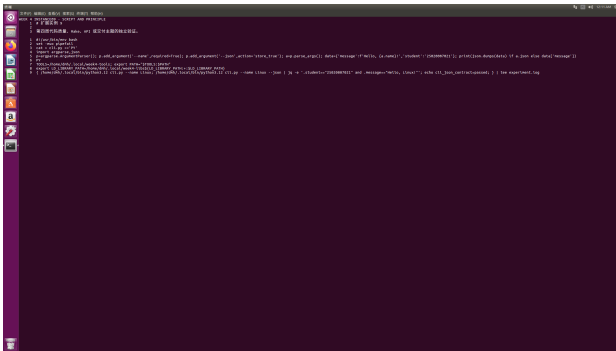
实例 8: jq 聚合命令



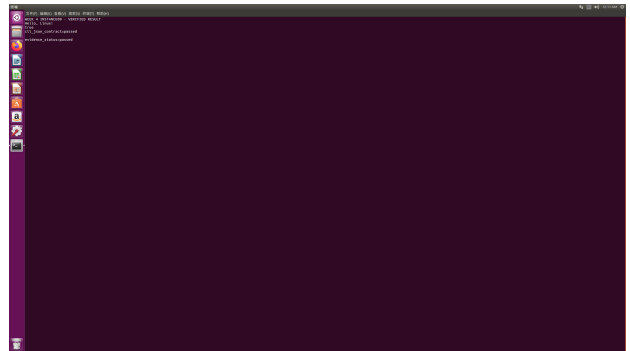
实例 8: JSON 统计结果

6.9 实例 9: CLI JSON 输出契约

CLI 同时支持文本与 JSON 输出；使用 jq 断言学号和消息字段，返回 true 并显示契约检查通过。首次使用系统默认 Python 出现 f-string 语法错误，改为 Python 3.12 后通过。



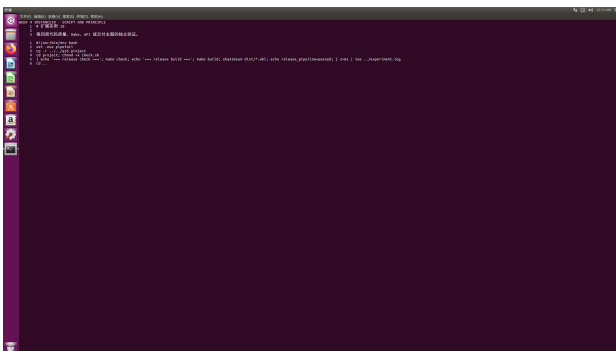
实例 9: CLI 脚本



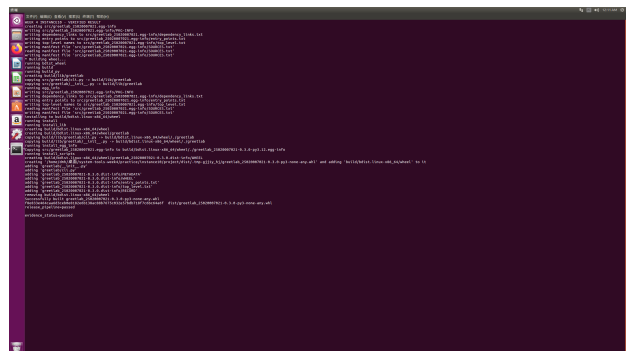
实例 9: jq 契约通过

6.10 实例 10: 完整发布流水线

复用 q16 项目，依次执行质量检查、wheel 构建和摘要计算；2 个测试、ruff 及构建全部通过，最终输出 `release_pipeline=passed`。



实例 10: 发布流水线



实例 10: 检查、构建与摘要

7 GitHub 分阶段提交记录

仓库地址为<https://github.com/dd666j/system-tools-week4>。课堂题 13-14、15-16 分别提交；扩展实例每两个一次提交，最后单独提交记录文件。主要提交为 92f93ad、18cef9f、065841b、8c994fd、ef775de、073fe9c、0de0300 和 1c33c63。

```
Powershell - Week 4 Git Commit Record

WEEK 4 COMMIT RECORD - 课堂题13-14
92f93ad 完成第四周课堂题13和14
q13/check.sh | 8 ++++++
q13/experiment.log | 13 ++++++
q13/pyproject.toml | 14 ++++++
q13/run_experiment.sh | 12 ++++++
q13/screenshots/q13_01_config_tests.png | Bin 0 -> 25528 bytes
q13/screenshots/q13_02_quality_gate.png | Bin 0 -> 139637 bytes
q13/src/gremlab/__init__.py | 1 +
q13/src/gremlab/cli.py | 18 ++++++
q13/test/test_cli.py | 9 ++++++
q14/Makefile | 3 +++
q14/build_report.py | 4 +++
q14/data.csv | 20 ++++++
q14/experiment.log | 1 +
q14/report.md | 1 +
q14/run_experiment.sh | 11 ++++++
q14/screenshots/q14_01_makefile.png | Bin 0 -> 13384 bytes
q14/screenshots/q14_02_incremental_result.png | Bin 0 -> 143488 bytes
q14/stats.py | 0 ++++++
18 files changed, 134 insertions(+)

originhttps://github.com/dd666j/system-tools-week4.git (fetch)
originhttps://github.com/dd666j/system-tools-week4.git (push)
```

课堂题 13-14 提交

```
Powershell - Week 4 Git Commit Record

WEEK 4 COMMIT RECORD - 课堂题15-16
18cef9f 完成第四周课堂题15和16
q15/api_report.sh | 16 ++++
q15/experiment.log | 15 ++++
q15/packages.json | 7 +++
q15/run_experiment.sh | 15 ++++
q15/screenshots/q15_01_api_pipeline.png | Bin 0 -> 254844 bytes
q15/screenshots/q15_02_summary_result.png | Bin 0 -> 122378 bytes
q15/server.log | 0
q15/summary.md | 7 +++
q15/Makefile | 8 ++
q16/check.sh | 0 +++
.../gremlab-25420087021-0.3.0-py3-none-any.whl | Bin 0 -> 1902 bytes
q16/experiment.log | 90 ++++++
q16/failure.log | 26 ++++++
q16/pyproject.toml | 14 ++++
q16/run_experiment.sh | 18 ++++++
q16/screenshots/q16_01_failure_fix.png | Bin 0 -> 175525 bytes
q16/screenshots/q16_02_release_result.png | Bin 0 -> 217794 bytes
q16/src/gremlab/__init__.py | 1 +
q16/src/gremlab/cli.py | 12 ++++
q16/test/test_cli.py | 12 ++++
28 files changed, 252 insertions(+)

originhttps://github.com/dd666j/system-tools-week4.git (fetch)
originhttps://github.com/dd666j/system-tools-week4.git (push)
```

课堂题 15-16 提交

```
Powershell - Week 4 Git Commit Record

WEEK 4 COMMIT RECORD - 实例1-02
065841b 完成第四周扩展实例1和实例2
practice/instance01/README.md | 3 +++
practice/instance01/after.txt | 1 +
practice/instance01/before.txt | 10 ++++++
practice/instance01/experiment.log | 16 ++++++
practice/instance01/run_experiment.sh | 7 ++++++
practice/instance01/sample.py | 2 ++
.../screenshots/instance01_01_process.png | Bin 0 -> 128919 bytes
.../screenshots/instance01_02_result.png | Bin 0 -> 109194 bytes
practice/instance02/README.md | 3 +++
practice/instance02/app.py | 2 ++
practice/instance02/before.txt | 16 ++++++
practice/instance02/experiment.log | 22 ++++++
practice/instance02/run_experiment.sh | 7 ++++++
.../screenshots/instance02_01_process.png | Bin 0 -> 126991 bytes
.../screenshots/instance02_02_result.png | Bin 0 -> 126991 bytes
15 files changed, 89 insertions(+)

originhttps://github.com/dd666j/system-tools-week4.git (fetch)
originhttps://github.com/dd666j/system-tools-week4.git (push)
```

实例 1-2 提交

```
Powershell - Week 4 Git Commit Record

WEEK 4 COMMIT RECORD - 实例3-04
8c994fd 完成第四周扩展实例3和实例4
practice/instance03/README.md | 8 +++
practice/instance03/experiment.log | 2 ++
practice/instance03/normalize.py | 5 ++++++
practice/instance03/run_experiment.sh | 19 ++++++
.../instance03/screenshots/instance03_01_process.png | Bin 0 -> 335821 bytes
.../instance03/screenshots/instance03_02_result.png | Bin 0 -> 87636 bytes
practice/instance03/test_normalize.py | 7 ++++++
practice/instance04/Makefile | 0 ++++++
practice/instance04/README.md | 3 +++
practice/instance04/experiment.log | 8 ++++++
practice/instance04/run_experiment.sh | 14 ++++++
.../instance04/screenshots/instance04_01_process.png | Bin 0 -> 128554 bytes
.../instance04/screenshots/instance04_02_result.png | Bin 0 -> 99191 bytes
practice/instance04/source.txt | 1 +
practice/instance04/supper.txt | 1 +
15 files changed, 69 insertions(+)

originhttps://github.com/dd666j/system-tools-week4.git (fetch)
originhttps://github.com/dd666j/system-tools-week4.git (push)
```

实例 3-4 提交

```
Powershell - Week 4 Git Commit Record

WEEK 4 COMMIT RECORD - 实例5-06
ef775de 完成第四周扩展实例5和实例6
practice/instance05/Makefile | 3 +++
practice/instance05/README.md | 3 +++
practice/instance05/data.txt | 10 ++++++
practice/instance05/experiment.log | 2 ++
practice/instance05/run_experiment.sh | 12 ++++++
.../instance05/screenshots/instance05_01_process.png | Bin 0 -> 127874 bytes
.../instance05/screenshots/instance05_02_result.png | Bin 0 -> 102687 bytes
practice/instance05/title.txt | 1 +
practice/instance06/Makefile | 8 ++++++
practice/instance06/README.md | 1 +
practice/instance06/a.out | 1 +
practice/instance06/b.out | 1 +
practice/instance06/experiment.log | 13 ++++++
practice/instance06/run_experiment.sh | 7 ++++++
.../instance06/screenshots/instance06_01_process.png | Bin 0 -> 127228 bytes
.../instance06/screenshots/instance06_02_result.png | Bin 0 -> 95544 bytes
18 files changed, 71 insertions(+)

originhttps://github.com/dd666j/system-tools-week4.git (fetch)
originhttps://github.com/dd666j/system-tools-week4.git (push)
```

实例 5-6 提交

```
Powershell - Week 4 Git Commit Record

WEEK 4 COMMIT RECORD - 实例7-08
073fe9c 完成第四周扩展实例7和实例8
practice/instance07/README.md | 3 +++
practice/instance07/data.json | 1 +
practice/instance07/experiment.log | 5 ++++++
practice/instance07/run_experiment.sh | 6 ++++++
.../instance07/screenshots/instance07_01_process.png | Bin 0 -> 127882 bytes
.../instance07/screenshots/instance07_02_result.png | Bin 0 -> 96593 bytes
practice/instance07/server.log | 0
practice/instance08/README.md | 3 +++
practice/instance08/experiment.log | 5 ++++++
practice/instance08/packages.json | 1 +
practice/instance08/run_experiment.sh | 8 ++++++
.../instance08/screenshots/instance08_01_process.png | Bin 0 -> 132844 bytes
.../instance08/screenshots/instance08_02_result.png | Bin 0 -> 91581 bytes
13 files changed, 32 insertions(+)

originhttps://github.com/dd666j/system-tools-week4.git (fetch)
originhttps://github.com/dd666j/system-tools-week4.git (push)
```

实例 7-8 提交

```
PowerShell - Week 4 Git Commit Record

WEEK 4 COMMIT RECORD - 实例9-10
0de0300 完成第四周广南实例9和实例10
practice/instance09/README.md | 3 +
practice/instance09/cil.py | 2 +
practice/instance09/experiment.log | 3 +
practice/instance09/run_experiment.sh | 9 +++
.../screenshots/instance09_R2_process.png | Bin 0 -> 137884 bytes
.../screenshots/instance09_R2_result.png | Bin 0 -> 89154 bytes
practice/instance10/README.md | 3 +
practice/instance10/experiment.log | 61 ++++++
practice/instance10/project/Makefile | 8 ++
practice/instance10/project/check.sh | 6 ++
.../greetlab_25828087821-0.3.8-py3-none-any.whl | Bin 0 -> 1502 bytes
practice/instance10/project/experiment.log | 50 ++++++
practice/instance10/project/failure.log | 26 +++++
practice/instance10/project/pyproject.toml | 14 +++
practice/instance10/project/run_experiment.sh | 48 +++++
.../project/screenshots/qd6_R3_failure_fix.png | Bin 0 -> 175525 bytes
.../project/screenshots/qd6_R2_release_result.png | Bin 0 -> 217794 bytes
.../instance10/project/src/greetlab/_init_.py | 1 +
practice/instance10/project/src/greetlab/cil.py | 17 +++
practice/instance10/project/test/test_cil.py | 12 +++
practice/instance10/run_experiment.sh | 6 ++
.../screenshots/instance10_R2_process.png | Bin 0 -> 114122 bytes
.../screenshots/instance10_R2_result.png | Bin 0 -> 280214 bytes
23 files changed, 279 insertions(+)

originhttps://github.com/dd666j/system-tools-week4.git (fetch)
```

实例 9-10 提交

```
PowerShell - Week 4 Git Commit Record

WEEK 4 COMMIT RECORD - 初始化仓库
a074619 初始化第四周实验仓库
README.md | 6 +++++
1 file changed, 6 insertions(+)

originhttps://github.com/dd666j/system-tools-week4.git (fetch)
originhttps://github.com/dd666j/system-tools-week4.git (push)
```

仓库初始化与完整历史

8 综合结论

本周完成了从代码质量、自动测试到增量构建、API 数据处理和 wheel 交付的完整工具链。实验中遇到的主要问题是默认 Python 版本不兼容、虚拟机缺少 curl 和 jq、功能失败被格式门禁提前遮蔽。解决原则是保留失败证据、定位实际失败层级、显式固定运行时与工具路径，并用最小修改重新验证。最终所有课堂题和 10 个扩展实例均在 Linux 实测通过，源码、日志、截图和 Git 历史已同步到 Windows 与公开 GitHub 仓库。